

Trait — Từ Bản Chất Đến Nâng Cao

Trait là **nền tảng** của polymorphism trong Rust. Hiểu trait → hiểu generic, closure, async, error handling, iterators. File này đi từ vấn đề căn bản (vì sao cần trait?), qua quy tắc và pattern, đến những góc sâu nhất (vtable, coherence, GAT).

Mục lục

Tầng 1 — Vì sao Trait tồn tại?

1. Bài toán polymorphism trong mọi ngôn ngữ
2. Các cách giải quyết trước Rust
3. Trait là gì — định nghĩa cốt lõi

Tầng 2 — Trait cơ bản 4. Khai báo & implement trait 5. Default method 6. Trait inheritance — supertrait 7. Associated functions vs methods

Tầng 3 — Hai cách dùng Trait 8. Static dispatch — generic + impl Trait 9. Dynamic dispatch — dyn Trait 10. Trait object & object safety 11. Khi nào dùng cái nào?

Tầng 4 — Trait & Memory Model 12. Monomorphization — bản chất 13. vtable — bản chất 14. Fat pointer của trait object 15. Cost của static vs dynamic

Tầng 5 — Trait nâng cao 16. Associated types 17. Generic parameters trên trait 18. Associated type vs generic — khi nào dùng? 19. Where clauses 20. Marker traits — Send, Sync, Copy

Tầng 6 — Trait Coherence 21. Orphan rule 22. Newtype pattern — vượt qua orphan 23. Blanket implementation 24. Coherence & specialization

Tầng 7 — Pattern thực dụng 25. From/Into & TryFrom/TryInto 26. Deref & DerefMut 27. Drop trait 28. Iterator — trait quan trọng nhất 29. Display vs Debug 30. PartialEq, Eq, Hash, Ord

Tầng 8 — Trait nâng cao sâu 31. GAT — Generic Associated Types 32. Trait alias & trait combinations 33. Auto traits 34. Higher-Ranked Trait Bounds (HRTB) 35. Dyn-compatible (object-safe) rules sâu 36. Trait Upcasting

Tầng 9 — Bẫy & cách đọc lỗi 37. Lỗi trait thường gặp

TẦNG 1 — VÌ SAO TRAIT TỒN TẠI?

1. Bài toán polymorphism

Polymorphism = "nhiều hình thái" — viết 1 đoạn code làm việc với **nhiều kiểu khác nhau**.

Ví dụ bài toán

Bạn muốn viết hàm `print` in ra một giá trị. Bạn muốn nó work cho:

- `i32` (in số)
- `String` (in chuỗi)
- `Vec<i32>` (in list)
- Tùy chọn của tương lai (User định nghĩa)

```
fn print(x: ???) {
    // làm sao biết cách in ra?
}
```

→ Cần một cách để **trừu tượng hóa** "khả năng in".

Hai loại polymorphism

1. AD-HOC polymorphism (overloading)

Cùng tên function, signature khác nhau theo type

C++: `void print(int); void print(string); void print(Vec);`

→ Compiler chọn đúng phiên bản theo argument type

2. PARAMETRIC polymorphism (generic)

1 function làm việc với MỌI type, vẫn cùng code

Java: `<T> void print(T x) { ... }`

Rust: `fn print<T>(x: T) { ... }`

→ Code chỉ viết 1 lần

→ Rust kết hợp **cả 2** thông qua **trait**.

2. Các cách giải quyết

Cách 1: C — không có gì

```
void print_int(int x) { ... }
void print_str(char* s) { ... }
void print_vec(int* arr, int len) { ... }
// ... viết tay tất cả
```

→ Lặp code, không scale.

Cách 2: C++ — overloading + templates

```
template<typename T>
void print(T x) {
    std::cout << x;          // hy vọng T có operator<<
}
```

ƯU: Generic, compile-time

NHƯỢC:

- Lỗi compile rất khó đọc (templates instantiation deep)
- Không có "interface contract" rõ ràng
- Phải đọc body để biết T cần có gì

Cách 3: Java — Interface

```
interface Printable {
    void print();
}

class MyClass implements Printable {
    public void print() { ... }
}

void doPrint(Printable p) {
    p.print();
}
```

ƯU: Có contract rõ ràng

NHƯỢC:

- Phải implement trong file của class
- Không thể thêm interface cho class có sẵn (vd: Integer)
- Mọi đa hình → dynamic dispatch (chậm)
- Mọi object có vtable pointer (tốn RAM)

Cách 4: Haskell — Type class

```
class Show a where
    show :: a -> String

instance Show Int where ...
instance Show String where ...
```

→ Khái niệm: implement của type vs trait là **2 file riêng** → có thể thêm trait cho type có sẵn!

Rust = Type class kiểu Haskell, với syntax C++.

Cách 5: Rust — Trait

```
trait Print {
    fn print(&self);
}

impl Print for i32 {
    fn print(&self) { println!("{}", self); }
}

impl Print for String {
    fn print(&self) { println!("{}", self); }
}

fn do_print<T: Print>(x: T) {
    x.print();
}
```

ƯU:

- Contract rõ ràng (trait)
- Có thể implement bên ngoài type (orphan rule kiểm soát)
- 2 lựa chọn: static (zero-cost) vs dynamic (linh hoạt)
- Lỗi compile dễ đọc (so với C++ templates)

NHƯỢC:

- Học khó hơn Java interface
- Object-safe rules phức tạp

3. Trait là gì

Định nghĩa chính xác

Trait là một **contract** (hợp đồng) mô tả "tập hợp các method một type phải có". Type có thể tự nhận trait (implement). Hàm có thể **yêu cầu** type implement trait nào đó.

Mô hình tư duy

Trait = "khả năng"

Trait Display	≈	"có thể in dạng người đọc được"
Trait Iterator	≈	"có thể duyệt từng phần tử"
Trait Clone	≈	"có thể tạo bản sao"
Trait Send	≈	"có thể chuyển giữa thread"
Trait Sync	≈	"có thể chia sẻ giữa thread"

```
Type = "danh từ"
String, Vec, i32, Box, ...

>Type X có trait Y" = "X có khả năng Y"
```

So sánh với OOP

OOP (Java):

```
class Dog extends Animal
    implements Bark, Run
```

Quan hệ "IS-A" (kế thừa)

Rust:

```
struct Dog { ... }
impl Bark for Dog { ... }
impl Run for Dog { ... }
```

Quan hệ "HAS CAPABILITY"
(composition + traits)

→ Rust **không có inheritance** giữa struct. Chỉ có trait + struct.

TẦNG 2 — TRAIT CƠ BẢN

4. Khai báo & implement

Khai báo trait

```
trait Animal {
    fn name(&self) -> String;
    fn sound(&self) -> String;
}
```

Implement cho struct

```
struct Dog { name: String }

impl Animal for Dog {
    fn name(&self) -> String {
        self.name.clone()
    }
    fn sound(&self) -> String {
        String::from("Woof!")
    }
}
```

Gọi method

```
let d = Dog { name: "Rex".into() };  
println!("{}", d.name(), d.sound());
```

Quy tắc cú pháp

```
trait NAME [<GENERICS>] [: SUPERTRAIT + SUPERTRAIT] [where ...] {  
    fn METHOD(&self [, args]) [-> RET];  
  
    fn METHOD_WITH_DEFAULT(&self) -> ... {  
        // body  
    }  
  
    type ASSOCIATED;  
  
    const CONSTANT: TYPE;  
}  
  
impl [<GENERICS>] NAME [<TYPES>] for TYPE [where ...] {  
    fn METHOD(&self, ...) -> ... {  
        // body  
    }  
  
    type ASSOCIATED = ACTUAL_TYPE;  
  
    const CONSTANT: TYPE = VALUE;  
}
```

5. Default method

Trait có thể cung cấp **implementation mặc định**:

```
trait Greet {  
    fn name(&self) -> String;  
  
    // method với body có sẵn  
    fn greet(&self) {  
        println!("Hello, {}!", self.name());  
    }  
}  
  
struct User { name: String }  
  
impl Greet for User {  
    fn name(&self) -> String { self.name.clone() }  
    // greet() được dùng mặc định!
```

```
}

let u = User { name: "Alice".into() };
u.greet(); // "Hello, Alice!"
```

Override default

```
impl Greet for User {
    fn name(&self) -> String { self.name.clone() }

    fn greet(&self) {
        println!("Xin chào, {}!", self.name()); // override
    }
}
```

Vì sao default method quan trọng?

Khi thêm method MỚI vào trait có sẵn:

Trait Iterator có 70+ method!

```
trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>; // required

    // 70+ default method xây trên next():
    fn map(...) { ... default impl ... }
    fn filter(...) { ... default impl ... }
    fn collect(...) { ... default impl ... }
    fn sum(...) { ... default impl ... }
    ...
}
```

→ Bạn chỉ cần impl 1 method `next()`, được 70+ method miễn phí!

6. Supertrait

Trait có thể "yêu cầu" trait khác phải implement trước.

```
trait Animal {
    fn name(&self) -> String;
}

trait Pet: Animal { // Pet "kế thừa" Animal
    fn owner(&self) -> String;
}
```

```

struct Dog { name: String, owner: String }

// Phải impl CẢ HAI
impl Animal for Dog {
    fn name(&self) -> String { self.name.clone() }
}

impl Pet for Dog {
    fn owner(&self) -> String { self.owner.clone() }
}

```

Độc supertrait

```

trait Pet: Animal
    _____
Độc: "Pet REQUIRES Animal"
= Type nào impl Pet thì PHẢI impl Animal trước

```

Nhiều supertrait

```

trait SuperPet: Animal + Display + Clone {
    fn fancy_intro(&self) {
        println!("I am {}", self);    // dùng Display
    }
}

```

7. Associated functions vs methods

```

trait Shape {
    // METHOD: nhận &self / &mut self / self
    fn area(&self) -> f64;

    // ASSOCIATED FUNCTION: không nhận self (giống static method)
    fn default_color() -> String {
        String::from("black")
    }

    // Constructor là ASSOCIATED FUNCTION
    fn unit() -> Self;
}

struct Circle { radius: f64 }

impl Shape for Circle {

```



```
fn area(&self) -> f64 { 3.14 * self.radius * self.radius }
fn unit() -> Self { Circle { radius: 1.0 } }

}

// Gọi
let c = Circle::unit();           // associated function
let a = c.area();                 // method
let col = Circle::default_color(); // associated function
```

```
METHOD      : đối tượng . method()
ASSOCIATED FN : Type :: function()
```

TẦNG 3 — HAI CÁCH DÙNG TRAIT

8. Static dispatch — generic + impl Trait

Generic function với trait bound

```
fn print_all<T: Animal>(animal: &T) {
    println!("{}", animal.name());
}
```

Đọc: "Cho mọi type T mà implement Animal, ..."

impl Trait syntax (đường tắt)

```
fn print_all(animal: &impl Animal) {
    println!("{}", animal.name());
}
```

Tương đương generic ở trên — nhưng ngắn hơn, không cần đặt tên T.

Bản chất: monomorphization

Khi gọi:

```
let d = Dog { ... };
let c = Cat { ... };

print_all(&d);
print_all(&c);
```

Compiler **clone** function ra cho từng type:

```
// SINH RA TỰ ĐỘNG:
fn print_all__Dog(animal: &Dog) {
    println!("{}", animal.name());
}
fn print_all__Cat(animal: &Cat) {
    println!("{}", animal.name());
}
```

→ Mỗi call → call function "đúng type". Không có runtime overhead.

→ Gọi là **static dispatch** vì compiler **biết trước** sẽ call function nào (resolve tại compile).

9. Dynamic dispatch — **dyn Trait**

Khi nào cần?

```
// Bạn muốn list các animals khác nhau:
let animals: Vec<???> = vec![
    Dog { ... },
    Cat { ... },
    Bird { ... },
];
```

??? = gì? Không thể là **Vec<Dog>** (chỉ chứa Dog). Cần "type chung cho mọi Animal".

→ Dùng **dyn Animal**:

```
let animals: Vec<Box<dyn Animal>> = vec![
    Box::new(Dog { ... }),
    Box::new(Cat { ... }),
    Box::new(Bird { ... }),
];

for a in &animals {
    println!("{}", a.name()); // resolve tại RUNTIME
}
```

dyn Trait là gì?

dyn Trait là một **trait object** — abstract type "một cái gì đó implement Trait, chưa biết là gì".

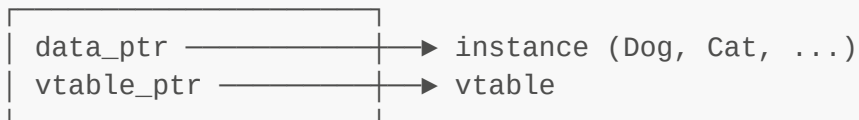
dyn Trait NOT là 1 type cụ thể.
Phải dùng qua POINTER:

- Box<dyn Trait>
- &dyn Trait
- Arc<dyn Trait>

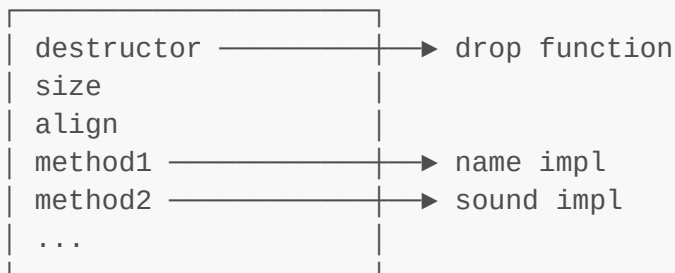
Vì sao? Vì compiler không biết SIZE của "thứ implement Trait"
(Dog 16 byte, Cat 24 byte, ...)
→ Phải lưu qua pointer.

Bản chất: vtable

Box<dyn Animal> thực ra là **fat pointer**:



vtable:



Khi gọi `a.name()`:

1. Lấy vtable_ptr từ fat pointer
2. Lookup name function trong vtable
3. Call function với data_ptr

→ **Runtime overhead**: 1 lookup trong vtable + 1 indirect call.

10. Trait object & object safety

Không phải trait nào cũng dùng được dưới dạng `dyn Trait`. Trait phải **object-safe** (hay "dyn-compatible").

Quy tắc object safety

Một trait là object-safe nếu **tất cả method** thỏa:

1. Method KHÔNG dùng generic type parameter
 - ✓ `fn foo(&self, x: i32)`

```
x fn foo<T>(&self, x: T)
```

Vì: dyn cần vtable cố định. Generic → vô số instantiation.

2. Method KHÔNG return Self (trừ khi qua pointer)

```
✓ fn foo(&self) -> i32
```

```
x fn clone(&self) -> Self
```

Vì: caller không biết size của Self.

3. Method KHÔNG có where clause `Self: Sized`

(Sized = "size biết lúc compile". dyn Trait không Sized.)

4. Method PHẢI có &self / &mut self / self (không phải associated function)

```
✓ fn area(&self)
```

```
x fn unit() -> Self
```

Ví dụ trait KHÔNG object-safe

```
trait Bad {
    fn clone(&self) -> Self;           // ✗ return Self
    fn process<T>(&self, x: T);        // ✗ generic
}

let x: Box<dyn Bad> = ...;            // ✗ ERROR
```

Workaround

```
trait Cloneable {
    fn clone_box(&self) -> Box<dyn Cloneable>; // ✓ qua pointer
}
```

11. Khi nào dùng cái nào?

STATIC DISPATCH

Cú pháp:

```
fn f<T: Trait>(x: T)
```

```
fn f(x: impl Trait)
```

Tốc độ:

DYNAMIC DISPATCH

Cú pháp:

```
fn f(x: Box<dyn Trait>)
```

```
fn f(x: &dyn Trait)
```

Tốc độ:

\$\$\$ (inline, optimize)

Code size:

📦 lớn (clone cho mọi type)

Compile time:

🐢 chậm hơn

Linh hoạt:

❌ phải biết type lúc compile

Use case:

- Function nhỏ, inline được
- Performance-critical
- Math, iterators

< (vtable lookup)

Code size:

📦 nhỏ (1 bản code)

Compile time:

🚀 nhanh hơn

Linh hoạt:

✓ runtime polymorphism

Use case:

- Plugin / heterogeneous list
- Lib không biết hết types
- GUI widget tree

Idiom thực dụng

Tier 1: Mặc định dùng generic / impl Trait (static)

Tier 2: Khi cần lưu nhiều type khác nhau → dyn

Tier 3: Khi không chắc → đo và quyết định

TẦNG 4 — TRAIT & MEMORY MODEL

12. Monomorphization — bản chất

Monomorphization = compiler **sao chép** generic function thành nhiều phiên bản cụ thể, mỗi phiên bản cho 1 type.

Trước monomorphization (code bạn viết)

```
fn double<T: std::ops::Add<Output = T> + Copy>(x: T) -> T {
    x + x
}

fn main() {
    let a = double(5i32);
    let b = double(3.14f64);
    let c = double(1u8);
}
```

Sau monomorphization (compiler sinh ra)

```
fn double__i32(x: i32) -> i32 { x + x }
fn double__f64(x: f64) -> f64 { x + x }
fn double__u8(x: u8) -> u8 { x + x }

fn main() {
    let a = double__i32(5);
    let b = double__f64(3.14);
    let c = double__u8(1);
}
```

→ Mỗi phiên bản dùng **CPU instruction phù hợp** (ADD cho i32, FADD cho f64...). Tối ưu tuyệt đối.

Cost: binary size

Mỗi instantiation = 1 bản copy code trong binary.

Nếu bạn có hàm 1000-line với 10 types khác nhau:

→ 10 bản trong binary

→ Tăng code size 10x

→ Có thể giảm cache hit (instruction cache đầy)

Đôi khi DYN nhanh hơn vì code nhỏ → fit cache tốt hơn!

"Code bloat" thực tế?

Hầu như không vấn đề với function nhỏ — vì sau monomorph, compiler **inline** vào caller, nên hàm "biến mất".

Vấn đề chỉ xảy ra với hàm lớn (vd: serde Deserialize) → có thể tăng binary lên hàng MB.

13. vtable — bản chất

vtable = **bảng pointer** chứa địa chỉ các method, được tạo TÍNH lúc compile.

Sinh vtable

Compiler thấy `dyn Animal` cho type `Dog` → tạo 1 vtable:

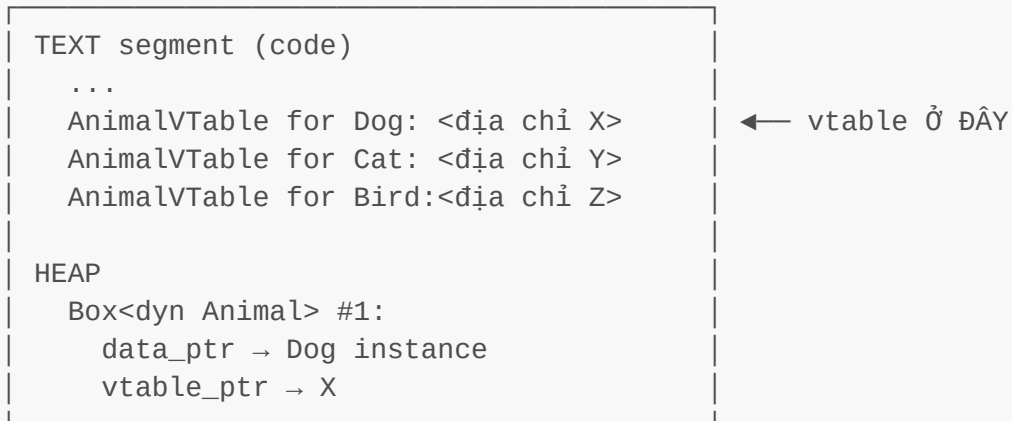
```
// Bạn viết:
let d: Box<dyn Animal> = Box::new(Dog { name: "Rex".into() });

// Compiler sinh:
static DOG_VTABLE_FOR_ANIMAL: AnimalVTable = AnimalVTable {
    destructor: drop_dog_in_place,
    size: 24, // sizeof(Dog)
    align: 8,
    name: Dog::name as fn(&Dog) -> String,
```

```
    sound: Dog::sound as fn(&Dog) -> String,
};
```

Lưu trữ vtable

Memory of program:



→ vtable là **static**, không nằm trên heap. Chỉ có 1 vtable per (Type, Trait).

Layout chi tiết của vtable

```

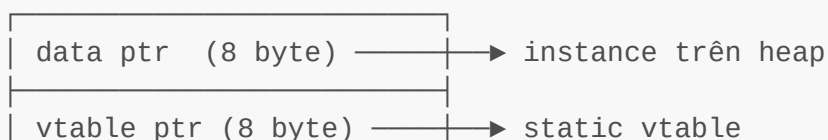
struct VTable {
    // Header chuẩn (mọi vtable đều có)
    destructor: fn(*mut ()),    // gọi khi drop
    size: usize,                // size của type
    align: usize,               // align của type

    // Method của trait
    method_0: fn(*const ()) -> ...,
    method_1: fn(*const ()) -> ...,
    ...
}
```

14. Fat pointer của trait object

`&dyn Trait` và `Box<dyn Trait>` là **fat pointer** — 2 word (16 byte trên 64-bit):

Box<dyn Animal>:



16 byte trên stack

So sánh với pointer thường

Box<Dog>:

ptr (8 B)

8 byte

Box<dyn Animal>:

data ptr (8 B)

vtable ptr (8 B)

16 byte (fat pointer)

Vì sao cần 2 pointer?

Box<Dog>:

- Đã biết là Dog → compiler biết:
 - * Size = 24
 - * Layout = (name: String)
 - * Methods = Dog::name (địa chỉ cứng)
- 1 pointer đủ.

Box<dyn Animal>:

- Không biết type cụ thể → compiler không biết:
 - * Size = bao nhiêu? (Dog 24, Cat 16)
 - * Methods ở đâu?
- Cần 2 pointer: data + metadata.

15. Cost comparison

Benchmark thực tế

```
trait DoStuff {
    fn work(&self, x: i32) -> i32;
}

// STATIC
fn run_static<T: DoStuff>(t: &T, x: i32) -> i32 {
    t.work(x)
}

// DYNAMIC
fn run_dynamic(t: &dyn DoStuff, x: i32) -> i32 {
    t.work(x)
}
```


Assembly sinh ra

```
run_static<Foo>:
    ; method inline thẳng vào đây nếu work() nhỏ
    add eax, ebx
    ret
                                                                    ← 1 instruction!

run_dynamic:
    mov rax, [rdi + 8]          ; load vtable ptr
    mov rax, [rax + 24]         ; load method ptr from vtable
    call rax                    ; indirect call
    ret
                                                                    ← ~3 instructions + branch

predictor miss
```

Thời gian thực tế (relative)

Operation	Time
Static dispatch (inlined)	~0.3 ns
Static dispatch (no inline)	~1.0 ns
Dynamic dispatch (vtable)	~2.0 ns
Function pointer call	~2.0 ns

- Dynamic chậm ~2-3x static, nhưng vẫn rất nhanh
- Đa số app KHÔNG cảm nhận khác biệt

Khi nào khác biệt quan trọng?

- Hot loop chạy hàng tỷ lần (game inner loop, ML)
- Branch predictor không đoán được (random type)

Khi đó → static rất đáng tiền.
Còn lại → dynamic OK.

TẦNG 5 — TRAIT NÂNG CAO

16. Associated types

Trait có thể chứa **type placeholder**:

```

trait Container {
    type Item;                // ← associated type

    fn get(&self, i: usize) -> Self::Item;
    fn len(&self) -> usize;
}

impl Container for Vec<i32> {
    type Item = i32;          // ← fill in

    fn get(&self, i: usize) -> i32 { self[i] }
    fn len(&self) -> usize { Vec::len(self) }
}

```

Vì sao cần associated type?

Vì 1 type chỉ impl trait **1 lần** với 1 Item duy nhất.

```

// Iterator trait:
trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
}

// Vec<i32>::iter() trả về Iterator với Item = &i32
// Vec<String>::iter() trả về Iterator với Item = &String

```

→ Item phụ thuộc vào type implement. Không phải bạn chọn lúc gọi.

17. Generic parameters trên trait

```

trait Convert<T> {
    fn convert(self) -> T;
}

impl Convert<String> for i32 {
    fn convert(self) -> String { self.to_string() }
}

impl Convert<f64> for i32 {
    fn convert(self) -> f64 { self as f64 }
}

```

→ 1 type có thể impl trait **nhều lần** với T khác nhau.

```
let x: i32 = 5;
let s: String = x.convert();           // Convert<String>
let f: f64 = x.convert();               // Convert<f64>
```

18. Associated type vs generic — khi nào dùng?

ASSOCIATED TYPE

```
trait Container {
    type Item;
    fn get(&self) -> Self::Item; }
}
```

1 type impl 1 LẦN duy nhất
với 1 Item duy nhất

Bind tightly
"Iterator này CHỈ trả về i32"

GENERIC PARAMETER

```
trait Convert<T> {
    fn convert(self) -> T;
```

1 type impl NHIỀU LẦN
với T khác nhau

Linh hoạt
"Có thể convert sang nhiều type"

Quy tắc thực dụng

- Nếu type X chỉ có 1 lựa chọn Item duy nhất:
→ Associated type
- Nếu type X có thể có NHIỀU output types:
→ Generic parameter

Ví dụ trong std

```
// Iterator: 1 Item duy nhất per type → associated
trait Iterator {
    type Item;
}

// From: nhiều conversion khác nhau → generic
trait From<T> {
    fn from(value: T) -> Self;
}

// Add: phổ biến nhất là 1 type, nhưng đôi khi cần đa dạng → cả 2!
trait Add<Rhs = Self> {
    type Output;
```

```
fn add(self, rhs: Rhs) -> Self::Output;
}
```

19. Where clauses

Khi trait bounds dài → tách ra **where**:

```
// Rườm rà:
fn process<T: Clone + Debug + Hash + Eq, U: Iterator<Item = T>>(x: U) { ... }

// Sạch:
fn process<T, U>(x: U)
where
    T: Clone + Debug + Hash + Eq,
    U: Iterator<Item = T>,
{ ... }
```

Where cho phép constraint không thể viết inline

```
fn foo<T>(x: T)
where
    Vec<T>: Clone, // ← không thể viết: T: ... gì cả
{ ... }
```

20. Marker traits — Send, Sync, Copy

Marker trait = trait không có method, chỉ để **đánh dấu** type có "tính chất" gì đó.

Send

```
unsafe trait Send {} // unsafe vì compiler không kiểm tra
```

T: Send = "có thể chuyển ownership của T sang thread khác".

Send:

i32, String, Vec<i32>
Box<T> nếu T: Send
...

NOT Send:

Rc<T> (counter không atomic)
*mut T (raw pointer)
MutexGuard<T> (lock specific thread)

Sync

```
unsafe trait Sync {}
```

T: Sync = "có thể chia sẻ &T giữa nhiều thread" (&T: Send).

Sync:

i32, String, Arc<T>
Mutex<T>, RwLock<T>

NOT Sync:

Cell<T>, RefCell<T>
Rc<T>

Auto traits

Send/Sync là **auto trait**: compiler tự suy diễn — nếu mọi field của struct là Send → struct là Send.

```
struct MyData {
    x: i32,           // Send
    y: String,        // Send
}
// MyData TỰ ĐỘNG Send.

struct WithRc {
    x: Rc<i32>,       // NOT Send
}
// WithRc tự động NOT Send.
```

Opt-out

```
struct ForceNotSend {
    x: i32,
    _phantom: PhantomData<*const ()>, // *const () NOT Send → opt-out
}
```

Copy / Clone

```
trait Copy: Clone {} // Copy là supertrait của Clone
trait Clone { ... }
```

Copy là marker (không method), Clone có method `clone()`.

Chỉ type "an toàn copy bằng memcpy" mới Copy:

- i32, f64, bool, char, (T1, T2) nếu T_i: Copy, [T; N] nếu T: Copy
- KHÔNG: String, Vec, Box, Rc, ...

TẦNG 6 — TRAIT COHERENCE

21. Orphan rule

Bạn **không thể** implement trait nào đó cho type nào đó tùy ý.

Quy tắc

Để `impl Trait for Type`, ÍT NHẤT MỘT trong:

- Trait được định nghĩa trong crate này
- Type được định nghĩa trong crate này

Ví dụ

```
// std::fmt::Display là external trait
// Vec là external type

impl std::fmt::Display for Vec<i32> { // ❌ ERROR
    fn fmt(...) { ... }
}
```

```
error[E0117]: only traits defined in the current crate can be
implemented for types defined outside of the crate
```

Vì sao quy tắc này tồn tại?

Giả sử:

- Crate A: `impl Display for Vec<i32> { ... }`
- Crate B: `impl Display for Vec<i32> { ... }`

User import cả A và B → impl nào được dùng?

→ AMBIGUITY → compile failure or worse, silent wrong behavior

→ Rust CẤM để mọi crate có thể compile cùng nhau an toàn.

Cho phép

```
impl MyTrait for Vec<i32> { ... }    // ✓ MyTrait local
impl Display for MyStruct { ... }   // ✓ MyStruct local
impl MyTrait for MyStruct { ... }   // ✓ cả hai local
```

22. Newtype pattern — vượt qua orphan

Muốn impl external trait cho external type? Wrap type vào struct mới:

```
struct MyVec(Vec<i32>);                // ← newtype

impl std::fmt::Display for MyVec {     // ✓ MyVec là local
    fn fmt(&self, f: &mut Formatter) -> fmt::Result {
        write!(f, "[{}]", self.0.iter().map(|i| i.to_string()).collect::<
<Vec<_>>().join(", ")))
    }
}
```

```
let v = MyVec(vec![1, 2, 3]);
println!("{}", v);    // "[1, 2, 3]"
```

Cost runtime

```
struct MyVec(Vec<i32>);
```

Layout: y hệt Vec<i32>, không thêm gì

- newtype = "zero-cost wrapper"
- Lúc runtime giống Vec<i32>.

Deref để truy cập method gốc

```
impl Deref for MyVec {
    type Target = Vec<i32>;
    fn deref(&self) -> &Vec<i32> { &self.0 }
}

let v = MyVec(vec![1, 2, 3]);
println!("{}", v.len());    // tự deref → Vec::len
```

23. Blanket implementation

Implement trait cho **mọi type** thỏa điều kiện.

Ví dụ trong std

```
// trong std:
impl<T: Display> ToString for T {
    fn to_string(&self) -> String {
        format!("{}", self)
    }
}
```

→ Mọi type có Display → tự động có `.to_string()`.

Mình tự định nghĩa

```
trait Square {
    fn square(&self) -> Self;
}

impl<T: Copy + std::ops::Mul<Output = T>> Square for T {
    fn square(&self) -> T {
        *self * *self
    }
}

let x = 5;
let y = 3.14;
println!("{}", x.square(), y.square());
```

Cẩn thận: conflict

Không thể có 2 blanket impl xung đột:

```
impl<T: Animal> Print for T { ... }
impl<T: Color> Print for T { ... }
```

Nếu có type vừa Animal vừa Color → ambiguity. Rust cấm.

24. Coherence & specialization

Coherence = quy tắc đảm bảo: với mỗi cặp (Trait, Type), có **đúng 1 impl** duy nhất trong toàn vũ trụ.

Coherence = Orphan rule + Overlapping impl rule

Orphan rule: kiểm soát impl ở crate nào
 Overlapping: cấm 2 impl chồng lên nhau

Specialization (nightly)

Trong nightly Rust, có feature `min_specialization` cho phép impl "specific hơn":

```
default impl<T> Display for Wrapper<T> { ... } // general
impl Display for Wrapper<i32> { ... }          // specialization
```

→ Cho `Wrapper<i32>` dùng impl thứ 2, mọi cái khác dùng default.

→ Vẫn chưa stable vì có nhiều case khó.

TẦNG 7 — PATTERN THỰC DỤNG

25. From / Into & TryFrom / TryInto

Cặp trait nền tảng nhất

```
trait From<T> {
    fn from(value: T) -> Self;
}

trait Into<T> {
    fn into(self) -> T;
}
```

Quy tắc đẹp

IMPLEMENT From — Rust TỰ ĐỘNG cho bạn Into!

```
impl From<i32> for MyNum { ... }
// → tự động có: impl Into<MyNum> for i32
```

Vì trong std:

```
impl<T, U: From<T>> Into<U> for T {
    fn into(self) -> U { U::from(self) }
}
```

→ Blanket impl tự generate Into từ From.

Ví dụ

```
let s = String::from("hi");           // From
let s: String = "hi".into();          // Into (cùng kết quả)

// Convert lỗi:
fn parse() -> Result<i32, MyError> {
    let n: i32 = "42".parse()
        .map_err(|e: ParseIntError| e.into()); // ParseIntError →
MyError
    Ok(n)
}
```

TryFrom / TryInto (có thể fail)

```
trait TryFrom<T> {
    type Error;
    fn try_from(value: T) -> Result<Self, Self::Error>;
}
```

```
let n: i32 = 256;
let b: u8 = n.try_into()?; // có thể fail nếu n > 255
```

26. Deref & DerefMut

Deref cho phép &T "tự động trở thành" &U:

```
trait Deref {
    type Target;
    fn deref(&self) -> &Self::Target;
}
```

Ví dụ kinh điển: Box, Rc, Arc

```
let b = Box::new(5);
println!("{}", *b); // dereference (gọi *b == *(b.deref()))
println!("{}", b);  // tự deref qua &i32 → fmt
```

Deref coercion — magic của Rust

Khi bạn pass `&String` đến hàm cần `&str`:

```
fn print(s: &str) { println!("{}", s); }

let s = String::from("hi");
print(&s);    // &String → &str tự động
```

Vì `String: Deref<Target = str>`.

Chain Deref tự động:

`&Box<String>` $\xrightarrow{\text{Deref}}$ `&String` $\xrightarrow{\text{Deref}}$ `&str`

Rust thử tất cả mức cho đến khi khớp signature.

Cảnh báo

Deref nên CHỈ dùng cho "smart pointer". Đừng abuse:

```
// Đừng làm:
impl Deref for User {
    type Target = String;
    fn deref(&self) -> &String { &self.name }
}

let u = User { name: ... };
println!("{}", u.len());    // confusing! User có .len() ở đâu?
```

27. Drop trait

```
trait Drop {
    fn drop(&mut self);
}
```

Đã giải thích chi tiết trong `ownership-borrowing.md`. Tóm tắt:

```
struct File { fd: i32 }

impl Drop for File {
    fn drop(&mut self) {
        // gọi syscall close(fd)
    }
}
```

→ Khi `File` instance hết scope, `close` tự gọi.

Quy tắc với Drop

1. Không thể gọi `.drop()` thủ công (chỉ system gọi)
2. drop được gọi NGAY CẢ KHI panic (unwinding)
3. Type có Drop KHÔNG thể move out field
(vì sẽ làm drop chạy 2 lần)

`std::mem::drop` ≠ `Drop::drop`

```
fn drop<T>(<_: T) {} // ← std::mem::drop, không có gì đặc biệt

let s = String::from("hi");
drop(s); // move s vào hàm, hàm end → s drop bình thường
```

28. Iterator — trait quan trọng nhất

```
trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;

    // ... 70+ default methods
    fn map<B, F>(self, f: F) -> Map<Self, F> where F: FnMut(Self::Item) -> B { ... }
    fn filter<P>(self, pred: P) -> Filter<Self, P> where P: FnMut(&Self::Item) -> bool { ... }
    fn collect<B>(self) -> B where B: FromIterator<Self::Item> { ... }
    // ...
}
```

Định nghĩa iterator của riêng bạn

```

struct Counter { count: u32 }

impl Iterator for Counter {
    type Item = u32;

    fn next(&mut self) -> Option<u32> {
        if self.count < 5 {
            self.count += 1;
            Some(self.count)
        } else {
            None
        }
    }
}

// Tự dùng có map, filter, sum, collect, ...
let total: u32 = Counter { count: 0 }
    .map(|x| x * 2)
    .filter(|x| x > &4)
    .sum();

```

Lazy evaluation

```

let v = vec![1, 2, 3, 4, 5];
let result: Vec<i32> = v.iter()
    .map(|x| { println!("map {}", x); x * 2 })
    .filter(|x| { println!("filter {}", x); x > &4 })
    .collect();

```

In ra:

```

map 1, filter 2,
map 2, filter 4,
map 3, filter 6,
map 4, filter 8,
map 5, filter 10

```

→ NOT: map tất cả → filter tất cả. Mà là: **mỗi item đi xuyên qua chain rồi mới đến item tiếp.**

→ Memory: không tạo Vec trung gian → cực kỳ tiết kiệm.

Zero-cost abstraction

```

let sum: i32 = (0..1000).filter(|x| x % 2 == 0).sum();

```

Sau optimize, code này nhanh ngang **for loop viết tay**. Compiler:

1. Monomorphize Iterator chain
2. Inline tất cả closure
3. Constant-fold nếu được
4. Vectorize (SIMD) nếu được

29. Display vs Debug

```
trait Display {
    fn fmt(&self, f: &mut Formatter) -> Result<(), Error>;
}

trait Debug {
    fn fmt(&self, f: &mut Formatter) -> Result<(), Error>;
}
```

Sự khác biệt

Display:

```
{ } trong println
"Cho người dùng"
PHẢI tự impl
"5"
"hello"
"[1, 2, 3]"

"Customer #5"
```

Debug:

```
{:?}", trong println
"Cho developer"
Có thể #[derive(Debug)]
"5"
"\hello\"
"[1, 2, 3]"

"Customer { id: 5, name: \"Alice\" }"
```

Pattern thực dụng

```
#[derive(Debug)] // luôn derive Debug
struct User { id: u32, name: String }

impl Display for User { // chỉ impl Display nếu cần show
    fn fmt(&self, f: &mut Formatter) -> std::fmt::Result {
        write!(f, "{} ({})", self.name, self.id)
    }
}
```

30. PartialEq, Eq, Hash, Ord

```

trait PartialEq {
    fn eq(&self, other: &Self) -> bool;
    fn ne(&self, other: &Self) -> bool { !self.eq(other) }
}

trait Eq: PartialEq {} // marker: equality là "totally" reflexive

trait PartialOrd: PartialEq {
    fn partial_cmp(&self, other: &Self) -> Option<Ordering>;
}

trait Ord: Eq + PartialOrd {
    fn cmp(&self, other: &Self) -> Ordering;
}

trait Hash {
    fn hash<H: Hasher>(&self, state: &mut H);
}

```

Vì sao PartialEq KHÁC Eq?

f64: PartialEq, NOT Eq

Vì NaN != NaN → vi phạm reflexivity!

Eq REQUIRES: $x == x$ luôn đúng.
f64 KHÔNG thỏa.

Khi nào dùng cái nào?

<code>#[derive(PartialEq)]</code>	cho mọi struct so sánh được
<code>#[derive(Eq)]</code>	thêm khi không có float
<code>#[derive(Hash)]</code>	cần làm HashMap key
<code>#[derive(PartialOrd, Ord)]</code>	cần sort

Quy tắc: PartialEq → Eq → Hash phải nhất quán

Nếu $x == y \rightarrow x.hash() == y.hash()$ (BẮT BUỘC!)

Nếu vi phạm → HashMap behavior undefined (mất key, key trùng, ...)

TẦNG 8 — TRAIT NÂNG CAO SÂU

31. GAT — Generic Associated Types

Associated type có lifetime / generic.

Trước GAT

```
trait Iterator {
    type Item; // Item không có lifetime
    fn next(&mut self) -> Option<Self::Item>;
}
```

Không thể trả về `&'a T` từ iterator (vì lifetime của ref phụ thuộc `&mut self`).

Sau GAT (Rust 1.65+)

```
trait LendingIterator {
    type Item<'a> where Self: 'a; // ← lifetime parameter!
    fn next<'a>(&'a mut self) -> Option<Self::Item<'a>>;
}

struct WindowsMut<'a, T> {
    slice: &'a mut [T],
    size: usize,
}

impl<'b, T> LendingIterator for WindowsMut<'b, T> {
    type Item<'a> = &'a mut [T] where Self: 'a;

    fn next<'a>(&'a mut self) -> Option<&'a mut [T]> {
        // ...
    }
}
```

→ Cho phép "iterator trả về reference vào chính self" — không khả thi với Iterator thường.

32. Trait combinations

`Trait1 + Trait2` ở bound

```
fn process<T: Display + Debug + Clone>(x: T) { ... }
```

`dyn Trait1 + Trait2` cho trait object


```
fn handle(x: &(dyn Display + Send)) { ... }
```

Auto trait bounds tự thêm

```
fn spawn(f: Box<dyn Fn() + Send + 'static>) { ... }
//           _____
//           Send   'static
//           (auto traits)
```

→ Rust **không** tự thêm Send/Sync vào `dyn Trait`. Bạn phải tự bound.

33. Auto traits

Auto trait có 4 cái chính:

- `Send`
- `Sync`
- `Unpin` (đối lập với `!Unpin` cho self-ref)
- `UnwindSafe` / `RefUnwindSafe`

"Auto" = compiler TỰ suy diễn cho mọi type.

Bạn không cần impl. Nếu mọi field thoả → struct cũng thoả.

Bạn cũng KHÔNG được impl thủ công (trừ nightly).

Opt out

```
struct ForceNotSend(*mut i32); // raw ptr không Send → struct không Send
```

```
struct ManualNotSend {
    _phantom: PhantomData<*const ()>,
}
```

34. Higher-Ranked Trait Bounds (HRTB)

Khi cần `Fn` work với "mọi lifetime":

```
fn apply<F>(f: F) where F: Fn(&str) -> bool { ... }
```

Liệu lifetime của `&str` là gì?

→ Compiler **tự thêm HRTB**: "for all lifetimes 'a":

```
fn apply<F>(f: F) where F: for<'a> Fn(&'a str) -> bool { ... }
//
//                                     HRTB
```

Khi nào tự viết?

99% code không cần. Chỉ khi compiler không suy ra được:

```
struct Processor<F> where F: for<'a> Fn(&'a [i32]) -> &'a i32 {
    f: F,
}
```

35. Dyn-compatible rules sâu

Đào sâu hơn về object safety.

Rule chi tiết

Một trait là object-safe khi:

1. Không là Sized supertrait
2. Mọi associated function (không có self) phải có where Self: Sized
3. Mọi method (có self) phải:
 - a. Không generic (trừ lifetimes)
 - b. Không return Self (hoặc dùng &Self/&mut Self/Box<Self>)
 - c. Không where Self: Sized (trong signature)
4. Không có associated const

Trick: rule 2 cho phép

```
trait Foo {
    fn helper() -> Self where Self: Sized;    // ← Self: Sized → bỏ qua
    object

    fn method(&self) -> i32;                  // object-safe method
}
```

```
// Foo VẪN object-safe!
// Vì helper() chỉ available khi gọi qua Type::helper(), không qua dyn
```

Workaround cho Self trong return

```
// ✗ Không object-safe:
trait Clone {
    fn clone(&self) -> Self;
}

// ✓ Object-safe wrapper:
trait CloneBox {
    fn clone_box(&self) -> Box<dyn CloneBox>;
}

impl<T: Clone + 'static> CloneBox for T {
    fn clone_box(&self) -> Box<dyn CloneBox> {
        Box::new(self.clone())
    }
}
```

36. Trait Upcasting

Từ Rust 1.86 (2024), có thể "upcast" trait object:

```
trait Animal { fn name(&self) -> String; }
trait Dog: Animal { fn bark(&self); }

fn process(d: &dyn Dog) {
    let a: &dyn Animal = d; // UPCAST – chỉ cần từ 1.86+
    println!("{}", a.name());
}
```

Trước 1.86 cần manual workaround. Giờ tự nhiên.

TẦNG 9 — BẦY & CÁCH ĐỌC LỖI

37. Common trait errors

Error 1: the trait X is not implemented for Y

```
error[E0277]: the trait bound `MyType: Display` is not satisfied
println!("{}", value);
      ^^^^^ the trait `Display` is not implemented for `MyType`
```

Fix: impl Display cho MyType, hoặc dùng `{:?}` (Debug).

Error 2: **the trait X cannot be made into an object**

```
error[E0038]: the trait `MyTrait` cannot be made into an object

let x: Box<dyn MyTrait> = ...;
      ^^^^^^^^^^^^^^^^^^^^^^^^^
```

Lý do: MyTrait có method với generic, return Self, hoặc **Self: Sized**. → đọc rule object-safe.

Error 3: **conflicting implementations**

```
error[E0119]: conflicting implementations of trait `MyTrait` for type `Foo`
```

Lý do: 2 impl chồng nhau. Có thể do blanket impl + specific impl.

Error 4: **only traits defined in current crate...**

Orphan rule. Dùng **newtype**.

Error 5: lifetime trong trait

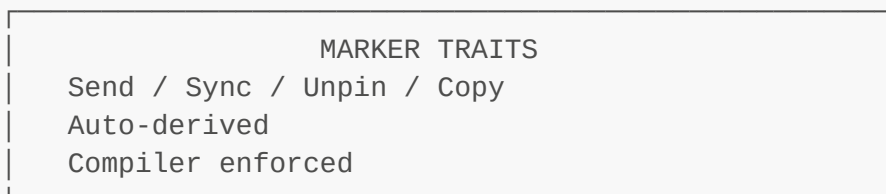
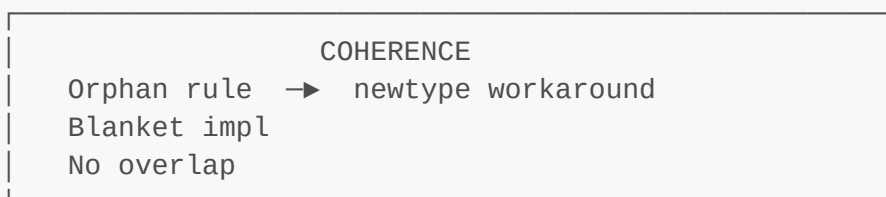
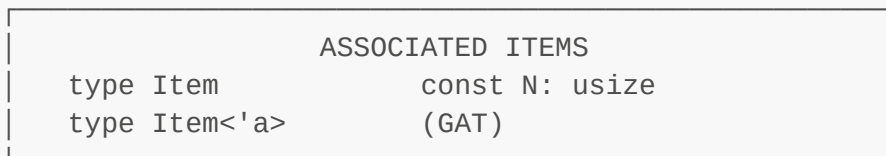
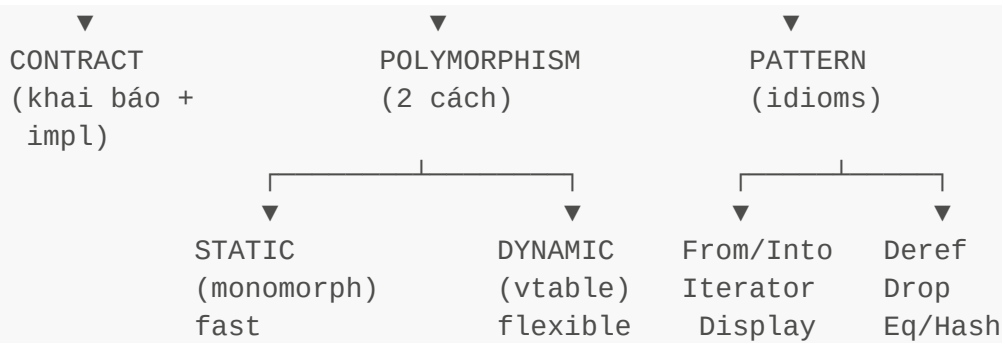
```
error: `impl` item signature doesn't match `trait` item signature
  expected `fn(&'a Self) -> &'a i32`
  found    `fn(&Self) -> &i32`
```

Lifetime trong impl phải khớp với trait.

KẾT LUẬN

Bản đồ tư duy





8 Quy luật ghi nhớ

1. Trait = contract về capability, không phải class
2. Type không inherit type, chỉ có trait
3. Generic = static dispatch = monomorphization (fast, big binary)
4. dyn Trait = dynamic dispatch = vtable (flex, small binary)
5. Object-safe trait mới dùng được dưới dạng dyn
6. Orphan rule: ít nhất Trait HOẶC Type phải local
7. Default method giúp trait scale (Iterator có 70+ method)
8. Marker trait (Send/Sync) là contract về memory/thread safety

Liên hệ Memory Model

Trait ↔ vtable trong TEXT segment, fat pointer 16 byte
 Generic ↔ monomorphization → multiple binary copies
 Iterator ↔ lazy, zero-copy chain, không tạo intermediate Vec
 Send/Sync ↔ contract về share/move giữa thread, builtin

```
Box<dyn>    ↔ heap allocation + vtable lookup  
Deref       ↔ tự deref qua nhiều cấp Box/Rc/Arc → cache-friendly
```

Lộ trình tiếp theo

Bây giờ bạn đã nắm trait → có thể học:

- **Generic** (xây trên trait bounds)
- **Closure** (Fn/FnMut/FnOnce là trait)
- **Async** (Future là trait)
- **Error handling** (? operator dùng From trait)
- **Iterator advanced** (combinators)

Đọc song song [trait-visual.md](#) để có hình minh họa trực quan từng phần.

Trait là "ngôn ngữ chính" Rust nói với compiler về capability. Hiểu trait sâu = hiểu cách Rust suy nghĩ.